PaperTrail — Project Documentation

Cryptographic Chain-of-Custody Audit Protocol for Tamper-Evident Government Document Tracking

Team: PaperTrail | Team ID: 56EGZG | Smart Kopargaon Hackathon 2026 | PS ID: SKH020

1. Project Overview

Project Name

PaperTrail — Chain-of-Custody Audit Protocol

Problem Statement

Government documents in India — examination papers, land mutation records, tender documents — pass through multiple custodians during their lifecycle (printing, transit, storage, distribution). At present there is no cryptographic mechanism to verify document integrity at each handoff point. When tampering occurs, it is impossible to determine exactly where in the chain of custody the alteration happened and under whose watch.

Three separate incidents within a single testing season in 2026 illustrate the scale of this gap. NEET-UG 2026 was cancelled on 12 May after a pre-circulated paper overlapped heavily with the actual exam; a CBI probe and raids followed across six states, and the Union Education Minister resigned on 25 July amid weeks of protest. In June 2026, Maharashtra TET papers were smuggled out of a printing press in Agra, concealed inside a worker's shoe, affecting over 6 lakh candidates across 1,028 centres. In the same window, a 100-page PDF of the UGC-NET Sociology paper was leaked and sold across five states before the exam, prompting the Ministry of Education to order an NTA investigation. Three different attack points — printing, transit, and digital circulation — and one shared root cause: once a document leaves its point of creation, nobody can prove whether it is still the original.

Proposed Solution

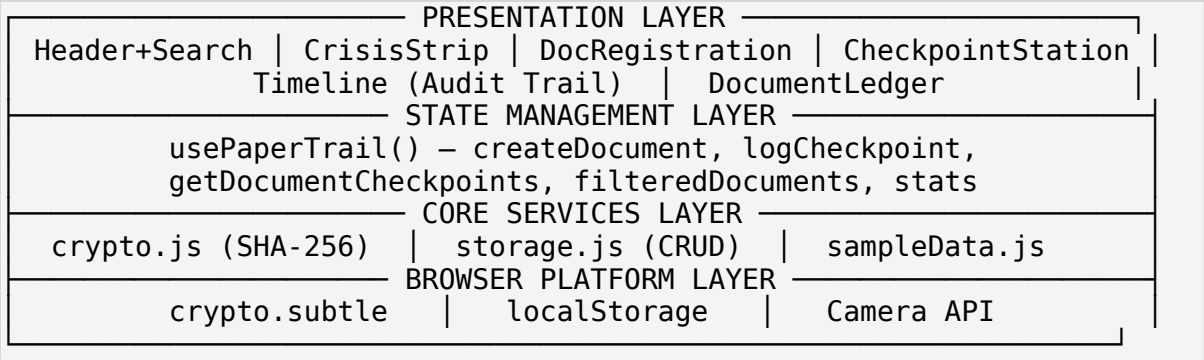
PaperTrail is a tamper-evident chain-of-custody system that assigns every document a SHA-256 cryptographic hash (digital fingerprint) at the moment of creation. This baseline hash is recomputed and compared at every subsequent handoff checkpoint. If even a single character of the document content is altered, the system immediately detects the mismatch, flags the exact checkpoint where tampering occurred, identifies the responsible custodian, and provides a side-by-side cryptographic diff showing the expected vs. received hash values.

Brief Technical Overview

The system is a client-side React web application using the browser's native Web Crypto API for SHA-256 computation. It has five core modules: (1) Crisis Dashboard, showing the verified real-world incidents above; (2) Genesis Vault, for document registration with initial hashing and QR generation; (3) Handoff Verifier, for checkpoint logging with dual-path document selection (QR scan or manual ID entry); (4) Immutable Audit Timeline, for visual chain inspection; and (5) Repository Ledger, with search, filtering, and JSON export. All data persists in browser localStorage, with the architecture designed for future extension to a blockchain-backed ledger.

2. System Architecture & Technology Stack

Architecture / Block Diagram



Key Modules

Module	Responsibility	File
Crypto Engine	SHA-256 hashing, ID generation, hash truncation	crypto.js
Persistence Layer	localStorage read/write, JSON serialization	storage.js
State Management	Centralized hook, single source of truth	usePaperTrail.js
UI Components	8 feature + 3 shared UI primitives	components/*.jsx
Sample Data Engine	Pre-computed walkthrough scenarios	sampleData.js

Technology Stack

Layer	Technology	Justification
UI Framework	React 19	Functional components, hooks-only
Styling	Tailwind CSS 4	Utility-first, custom design tokens
Build Tool	Vite	Fast HMR, optimized production build
Cryptography	Web Crypto API (native)	No third-party crypto dependency
QR Generation	qrcode.react	SVG-based QR rendering

Layer	Technology	Justification
QR Scanning	html5-qrcode	Camera-based checkpoint scanning
Persistence	localStorage (native)	Client-side JSON storage

System Requirements

Modern browser with Web Crypto API support (Chrome 37+, Firefox 34+, Edge 12+, Safari 11+). No server, no database, no internet connection required after initial page load.

Design Rationale — Why Not a Distributed Ledger

The chain-of-custody problem is deliberately solved with a hash chain rather than a blockchain. A blockchain's value is consensus among mutually distrusting parties with no central authority; PaperTrail's custodians are already known, authorized government staff within a single institutional workflow, so distributed consensus solves a problem that does not exist here while adding real cost: new infrastructure, transaction latency, and a technical barrier for non-technical staff. A hash chain — each record cryptographically referencing the one before it — gives the same tamper-evidence guarantee for this use case, runs entirely client-side on hardware departments already own, and keeps the operating interface as simple as scan and verify.

3. System Design

Data Models

```
Document: { id, title, category, createdAt, initialHash, currentStatus,
totalCheckpoints }
Checkpoint: { id, documentId, stageName, custodianName, custodianRole,
timestamp,
  contentSnapshot, computedHash, previousHash, status, tamperDetails }
```

Data Flow

```
REGISTRATION:  input → SHA-256(content) → Document{initialHash} →
Genesis
  Checkpoint{status:sealed} → QR generated → saved to localStorage

CHECKPOINT:  select document (QR / manual ID) → enter observed content
→
  SHA-256(content) → compare with previous checkpoint's hash →
    match     → status: verified
    mismatch  → status: tampered, store {expectedHash, receivedHash}
```

User Roles & Access Control

Single-role system for this prototype — any authorized custodian can register documents and log checkpoints. Role differentiation is recorded per checkpoint (custodian name and role) for audit purposes; access control is listed under Future Scope.

4. Technical Workflow & Methodology

End-to-End Workflow

```
[1] GENESIS SEALING → content hashed → QR generated → status SEALED
[2] CHECKPOINT (repeatable) → scan/enter ID → enter observed content
      → hash compared to previous checkpoint
      → match: VERIFIED (green)   |   mismatch: TAMPERED (red),
        shows expected vs. received hash, custodian, and stage
[3] AUDIT INSPECTION → vertical timeline per document, expandable nodes
[4] EXPORT → full audit trail downloadable as JSON
```

Core Algorithm — Hash Comparison

```
const hash = await generateSHA256(content);
const previousHash = lastCheckpoint?.computedHash;
if (!lastCheckpoint)            status = 'sealed';
else if (hash === previousHash) status = 'verified';
else                            status = 'tampered';
```

This is a strict equality check on 64-character hexadecimal strings. SHA-256's avalanche effect guarantees a single-bit change in input produces a completely different output, making undetected alteration computationally infeasible.

5. Implementation

Module-wise Implementation

Crypto Engine (crypto.js): uses crypto.subtle.digest('SHA-256', data) natively. Content is UTF-8 encoded via TextEncoder, hashed to a 256-bit buffer, converted to a 64-character hex string. No third-party crypto library — this removes a supply-chain attack surface. Document IDs follow DOC-MH-YYYY-XXXX; checkpoint IDs follow CHK-XXXXX.

Persistence (storage.js): two localStorage keys, papertrail_documents and papertrail_checkpoints, serialized as JSON. Reads are wrapped in try/catch, returning empty arrays on parse failure so corrupted storage cannot crash the app. Writes fire reactively via useEffect.

State Management (usePaperTrail.js): a single custom hook exposing createDocument, logCheckpoint, getDocumentCheckpoints, getDocument, resetAll, loadSample, plus computed filteredDocuments and stats. Search filters across document ID, title, status, custodian, and stage.

Document Registration: three document categories (Exam Paper, Land Mutation, Government Tender). Live SHA-256 preview as the user types. On submit, generates a QR

code (qrcode.react, error correction level H) encoding the document ID, and displays the full hash with a SEALED badge.

Checkpoint Station: dual-path document selection — QR scan (html5-qrcode, camera access) or manual ID entry with auto-suggest — presented as equally valid paths, not a hidden fallback. Four predefined custody stages plus a custom-stage option. Verification result shows VERIFIED (green) or TAMPERED (red, with side-by-side expected/received hash and custodian attribution).

Audit Timeline: vertical timeline, colour-coded nodes (sealed / verified / tampered), a broken-link icon at the point a chain fails. Each node expands to show the full hash, previous hash, and content snapshot.

Repository Ledger: responsive card grid — category, status, title, ID, creation date, checkpoint count, current stage. A JSON export button downloads the complete audit trail per document.

Key Technical Components

Component	Library	Purpose
<QRCodeSVG>	qrcode.react	SVG QR code for document ID
Html5QrcodeScanner	html5-qrcode	Camera-based checkpoint scanning
crypto.subtle.digest	Web Crypto API	Native SHA-256 computation
localStorage	Web Storage API	Persistent client-side storage

6. Testing & Validation

Testing Approach

Manual end-to-end testing across three scenarios — clean-chain verification, tamper detection, and data persistence — supplemented by isolated validation of the SHA-256 implementation against Node's native crypto module.

Test Cases

#	Test case	Expected outcome	Result
1	Genesis sealing	Document created, QR + hash + SEALED	Pass
2	Clean checkpoint	Status VERIFIED	Pass
3	Tampered checkpoint	Status TAMPERED, hash diff shown	Pass
4	Hash determinism	Identical input → identical hash	Pass

#	Test case	Expected outcome	Result
5	Avalanche effect	1-word change → wholly different hash	Pass
6	Persistence	Data survives browser refresh	Pass
7	Search filtering	Correct results by custodian/ID/stage	Pass
8	Sample data load	2 documents, 6 checkpoints populate	Pass
9	JSON export	Valid audit-trail JSON downloads	Pass

Validation Method

Functional correctness was validated by direct interaction with the running application against each test case below, rather than mocked unit tests, since the property under test — does a real content change produce a real hash mismatch — is only meaningful when exercised through the actual crypto.subtle call path the application uses in production.

Cryptographic Validation

Independently verified (Node.js crypto, sha256sum):

```
Input:    "This is the official test content for verification"
SHA-256:
a25cb81b41714aaaa2007c248ecc2e6b4649c962926a4839b1e3c2e2cce27ae8
Modified: "This is the MODIFIED test content for verification"
SHA-256:
628b33cafc646888b4dce328dabdcf921447ca9a6a1db6ffd2af27bef9dd94a0
Match: FALSE → tampering correctly detected.
```

Performance

Metric	Value
Production JS bundle	258 KB (79.5 KB gzip)
CSS bundle	26.5 KB (5.9 KB gzip)
Build time	1.19 s
SHA-256 computation	< 1 ms per hash (native Web Crypto)

7. Results, Limitations & Future Scope

Results

The prototype demonstrates tamper-evident tracking across a full custody lifecycle via two scenarios: an exam paper passing three checkpoints unaltered (all verified), and a land mutation record altered at the custodian level, which PaperTrail correctly localizes to the exact stage, custodian, and hash mismatch. The system supports arbitrary document types and an unlimited number of checkpoints per document.

Technical Limitations

(1) Client-side only — data lives in localStorage (~5–10 MB limit), no server backup or cross-device sync. This is acceptable for a single-department pilot but not for the multi-department Phase 2 rollout described in the original proposal. (2) No authentication — any user can currently register documents or log checkpoints; production deployment requires role-based access control tied to institutional identity. (3) Text-only hashing — binary files (PDF, image) need FileReader integration to hash the raw bytes rather than extracted text. (4) No encryption at rest — content is stored as plaintext in localStorage, so sensitive documents would need AES encryption before this moves past a demo. (5) Single-device scope — the chain-of-custody as implemented lives in one browser instance; multi-party verification across departments needs a shared backend or the blockchain-anchored variant described below.

Future Technical Improvements

Blockchain-backed ledger for decentralized, immutable checkpoint records; binary file hashing via FileReader.readAsArrayBuffer(); Ed25519 custodian signatures binding each checkpoint to an authorized individual; integration with DigiLocker and state revenue department APIs for automated ingestion; a mobile-first offline-capable PWA; and a server-synced architecture for multi-party, multi-device verification. A geo-spatial hashing extension is also planned for land-record boundary coordinates, to catch encroachment as well as document tampering.

8. References

Research & Standards

1. FIPS 180-4: Secure Hash Standard (SHA-256), NIST, 2015.
2. W3C Web Cryptography API Specification — w3.org/TR/WebCryptoAPI

APIs & Browser Standards

3. SubtleCrypto.digest() — MDN Web Docs
4. Web Storage API (localStorage) — MDN Web Docs
5. MediaDevices.getUserMedia() — MDN Web Docs

Libraries & Frameworks

6. React 19 Documentation — react.dev 7. Tailwind CSS v4 — tailwindcss.com/docs 8. Vite — vite.dev

9. qrcode.react — github.com/zpao/qrcode.react 10. html5-qrcode — github.com/mebjas/html5-qrcode 11. Lucide Icons — lucide.dev

Incident References (verified, 2026)

12. 2026 NEET-UG controversy — en.wikipedia.org/wiki/2026_NEET_controversy

13. Education Minister's resignation, 25 July 2026 — France 24

14. Maharashtra TET printing-press breach — Free Press Journal

15. UGC-NET paper leak, Ministry orders NTA probe — India TV News, 9 July 2026

16. The Public Examinations (Prevention of Unfair Means) Act, 2024, Government of India